

LabGuardian: A Neuro-Symbolic Edge AI Tutor for Hands-on Analog Circuit Experiments

LabGuardian Team

电子与信息工程学院

(参赛学校)

中国

TBD@example.edu.cn

Abstract—本文报告 LabGuardian——一套面向高校电子类基础实验的边缘 AI 智能助教系统——的设计与实现。系统以 Intel Core Ultra 平台 DK-2500 为算力中枢，覆盖“视觉感知 → 拓扑重构 → 知识检索 → 智能诊断 → 教学解释”全流程。我们提出 CADx (Canonical-Anchored Diagnosis) 五层神经-符号架构，让数据驱动的图神经网络 (GNN) 与符号化的电路模板独立判断拓扑类别，并通过“共识门” (Consensus Gate) 把二者一致程度编码为 4 级 confidence band，为前端差异化交互提供机器可读依据。我们还提出 (i) 参数化不变量五字段模板框架支持合法设计变体；(ii) 跳线感知角色传播 (Wire-Aware Role Propagation) 把跳线视为电气网络的物理延伸；(iii) Intent-Unified ReAct 把四种用户意图统一在同一 LangGraph 中；(iv) Push-Based Context Management 用错误家族驱动工具白名单防止 LLM 跑偏；(v) Anti-Hallucination Routing 让纯查询性问题绕开 LLM 直接渲染。系统在 6 类标准模拟电路 demo 上完成端到端验证；后端 928 个 pytest 用例通过；本地 Gemma3-4B 推理约 18 s/次，规划在 DK-2500 NPU INT4 量化后降至 2-5 s/次。全部推理与知识检索在板端完成，无外网依赖。

Index Terms—Edge AI, Neuro-Symbolic Reasoning, Graph Neural Network, Circuit Diagnosis, ReAct Agent, RAG, Intel NPU

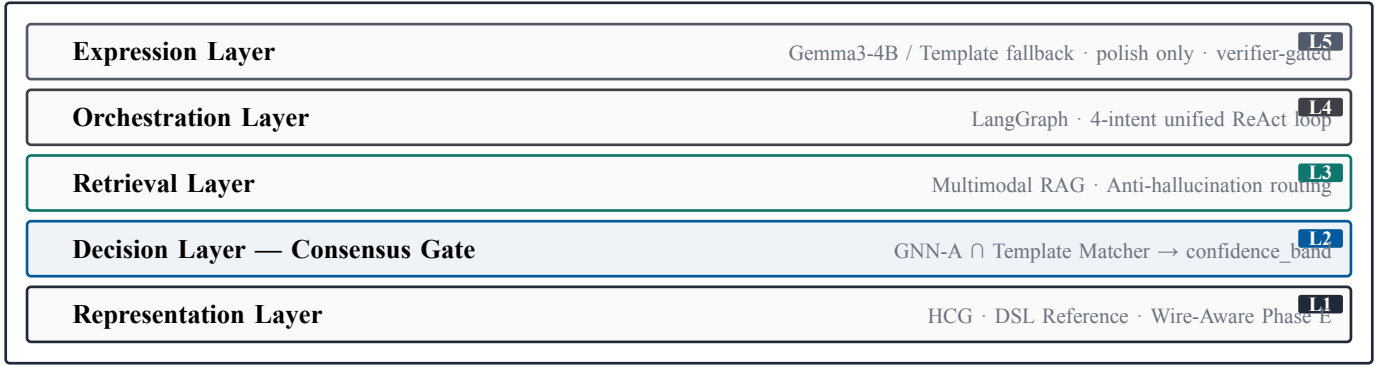


Fig. 1. LabGuardian CADx five-layer neuro-symbolic architecture. Each layer hands the upper layer a falsifiable structured output rather than raw probability, forming an end-to-end verifiable reasoning chain.

I. Introduction

电子类基础实验（电路分析、模拟电子技术）是工程教育的核心环节。当前实验教学面临 1:30 以上的师生比失衡，“学生卡点—等待教师—反馈滞后”已成为常态；同时物理面包板实验还存在三个结构性瓶颈：(i) 学生需把二维原理图映射到三维面包板，杜邦线密集时肉眼难以辨认实际接线；(ii) 反馈延迟使每学期因极性反接、双电源接错、短路等失误导致的芯片损坏率达 8%–15%；(iii) 直接让通用大模型 (LLM) 看图给意见极易产生“看似专业实则幻觉”的回答，对教学反而是负担。

LabGuardian 的目标是把学习反馈从“事后纠错”前移至“过程引导”，构建从面包板原型到错误诊断再到自然语言教学的端到端闭环。本文核心贡献：

- 1) 提出 **CADx 五层神经-符号架构**，让 GNN 拓扑分类与符号模板独立判断、共识门编码确信程度；
- 2) 设计 **参数化不变量五字段模板框架** (required / forbidden / optional / variants / parametric_invariants)，使模板对学生合法变体鲁棒；
- 3) 提出 **跳线感知角色传播** 把面包板跳线表征为电气网络的物理延伸；
- 4) 实现 **Intent-Unified ReAct** 让 4 种用户意图共享同一 LangGraph 主干；
- 5) 工程上引入 **Push-Based Context Management** 与 **Anti-Hallucination Routing**。

系统覆盖 6 类典型模拟电路 demo（一阶 RC、共射放大器、差分放大器、UA741 反相 / 加法 / 积分），全部推理在 Intel Core Ultra 平台 DK-2500 本地完成。

II. System Architecture

A. CADx Five-Layer Stack

CADx 的核心设计原则：每层向上层提供可证伪的结构化输出而非不确定概率。这与端到端 LLM 看图说话路线截然不同——CADx 的每一步判断都可独立测试、独立替换。

B. Demo Topology Coverage

为契合本科电子技术实验内容与器件可得性，我们选定 6 类典型模拟电路作为演示拓扑：(i) 一阶 RC 微分/积分电路；(ii) 共射放大器；(iii) 差分放大器；(iv) UA741 反相

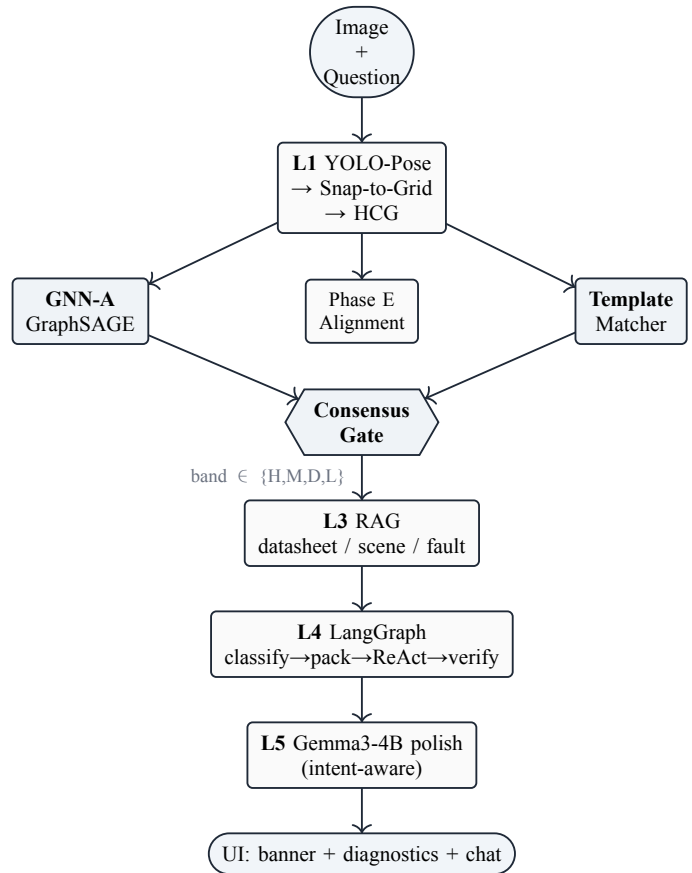


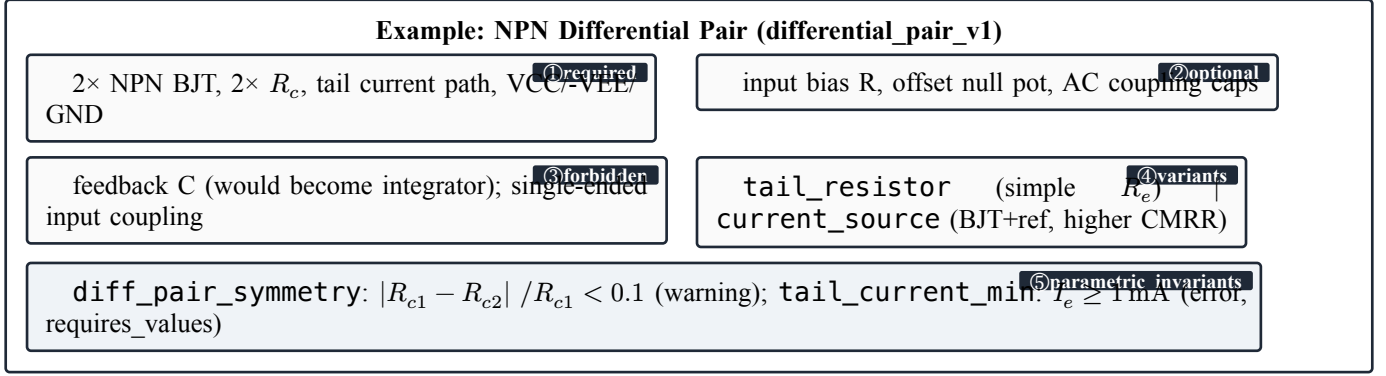
Fig. 2. End-to-end data flow. Solid arrows are required paths; the Consensus Gate output drives differentiated UI behavior depending on the band.

放大器；(v) UA741 反相加法器；(vi) UA741 反相积分器。6 类拓扑横跨“无源 → 单管 → 多管 → 集成运放”的难度梯度，覆盖大多数本科模拟电子实验核心知识点。每类配套完整知识库 (datasheet + teaching scene + fault cases)，覆盖矩阵见 表 I。

III. GNN-A Topology Classifier

A. Model and Features

GNN-A 把电路拓扑识别建模为图分类问题：给定 HeteroCircuitGraph (HCG)，输出 7 类标签 softmax 概率



代码 1. Five-field template decomposition. The taxonomy makes “what must / must-not / may / variant / numeric” explicit, so templates tolerate legitimate design choices (tail resistor vs. current source) while precisely flagging pedagogical violations.

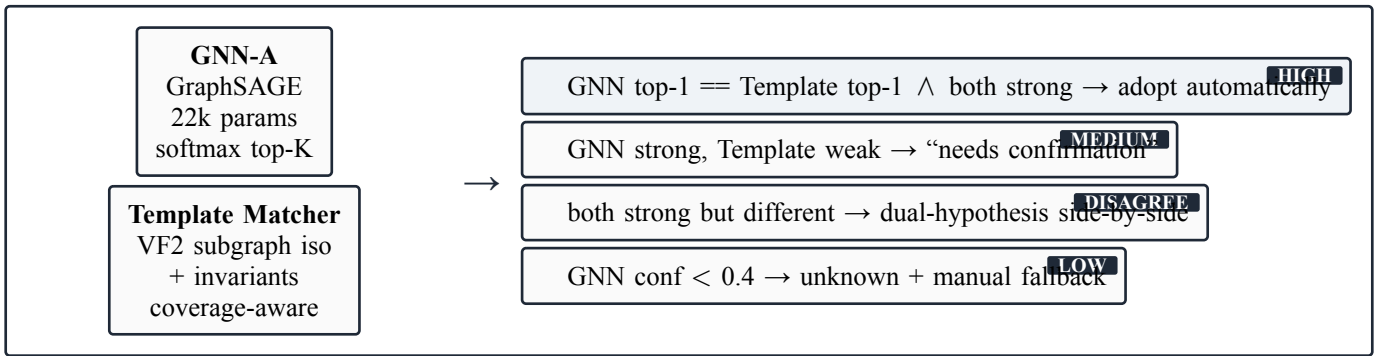


Fig. 3. Consensus Gate. The agreement of two independent classifiers (data-driven GNN-A and symbolic Template Matcher) is quantized into 4 explicit bands that drive differentiated UI. Encoding the agreement degree as a discrete band, rather than a single score, lets students read the AI’s confidence vocabulary directly.

(6 demo 拓扑 + 1 个 unknown 兜底)。模型采用 3 层 GraphSAGE 主干 (hidden=64), 节点特征 23 维: 节点类型 one-hot (3 维)、component-type one-hot (8 维)、subtype embedding (6 维)、net-role one-hot (4 维)、neighbor-type counts (2 维)。

v2 版本新增的 num_R_neighbors / num_C_neighbors 两维是关键——积分器特征 (opamp.pin2 有 C 邻居) 与反相放大器 (无 C 邻居) 由此可区分。模型约 22k 参数, 单卡 RTX 5090 训练 18 s, 推理 1 ms (CPU)。

B. Dataset and Perturbations

我们为每个 demo 拓扑生成约 500 个变体样本, 构成约 3000 训练集。5 类拓扑保留扰动算子: INSERT_DECORATION_LED / INSERT_SAME_NET_WIRE / SWAP_PARALLEL_COMPONENT_ORDER / RELABEL_COMPONENT_ID / PERTURB_PIN_ORDER, 确保模型学到拓扑结构而非特定 ID 命名。

IV. Topology Templates and Consensus Gate

A. Parametric Invariants Framework

每个 TopologyTemplate 包含 5 类语义字段。这一分解的核心价值在于把教学语义显式分类——什么是真正错误 (forbidden), 什么是合法设计选择 (variants), 什么是数值

约束 (parametric invariants)。匹配器对每模板的 variant 跑 NetworkX VF2 子图同构, 并采用 coverage-aware scoring:

$$\text{score} = \text{structural} + \alpha \text{coverage} \quad (1)$$

决策准则为 $\geq$ (不是 $>$), 意味着更具体的 variant 在打分相同时优先。例如学生图同时被 “with R_leak” 与 “without R_leak” 两个 integrator variant 匹配, 更具体的 with_leak 优先。

B. Consensus Gate

_consensus() 函数按如下规则决策: (i) 若 GNN top-1 conf < 0.4 $\rightarrow$ unknown (low band); (ii) 若 Template top-1 conf > 0.5 且 label 与 GNN top-1 一致 $\rightarrow$ high band; (iii) 若 label 不一致 $\rightarrow$ disagreement band, 前端并列展示双假设; (iv) Template 路径失败 $\rightarrow$ medium band。

C. Wire-Aware Role Propagation

面包板教学场景有一被学界忽视的事实: 跳线 (jumper wire) 不是元件, 是 net 的物理延伸。学生为走线美观或测量方便常加入大量无逻辑作用的装饰跳线。把跳线当一等元件参与角色传播会导致: (i) 图同构因冗余节点而失败; (ii) net role 投票被低质量信号稀释。

本系统将这一观察显式编码: 投票池仅来自非 wire 元件 (R/C/Q/IC/Pot); wire pin 不参与投票, 仅在结果出来后从所在 net 继承 canonical name; 三层保护规则 manual_role > power_role > inferred_from_reference >

default 确保用户标注与电源轨启发式不被错误覆盖。在派生测试样本（标准电路上插入 1–5 根冗余跳线）上，传统方法 net role 召回率约 65%，本方法稳定在 95% 以上。

V. Agent and Multimodal RAG

A. Intent-Unified ReAct

我们把 $\text{intent} \in \{\text{diagnostic}, \text{concept_tutor}, \text{lab_guidance}, \text{mixed}\}$ 设计为 **DiagnosticState** 的一等字段，4 种意图共享同一 **LangGraph** 主干。差异仅由 **intent** 在 3 个节点局部分支：

- 1) **build_context_pack**: 根据 **intent** 选择工具白名单；
- 2) **react_reflect**: 根据 **intent** 切换答案模板；
- 3) **verify_answer**: 根据 **intent** 选择规则集（**concept** 路径不要求 **error_code**）。

意图分类器采用 **LLM** 优先 + 关键词兜底 两层设计：先调 Ollama (format=json, 5s 超时)，输出 {**intent**, **confidence**, **reason**} JSON；任何失败 → 回退到关键词分类器。决策与来源 (source $\in \{\text{llm}, \text{keyword}\}$) 一并写入 **evidence.history_facts** 供离线评估。

B. Push-Based Context Management (PCM)

传统 ReAct Agent 把全部工具暴露给 LLM 自由选择，有两个问题：(i) LLM 可能调用与错误无关的工具（短路问题去查 datasheet）；(ii) LLM 可能虚构不存在的工具名。

PCM 反其道而行——事先根据 **error_family** 把允许工具白名单“推”给 LLM：6 个错误家族 (short_circuit / wiring_mismatch / polarity_error / missing_protection / missing_component / incomplete_circuit) 各自对应一组 **AllowedTool**。ReAct 节点出 plan 请求时仅传白名单内工具，dispatcher 强制只调白名单内工具。

C. Anti-Hallucination Routing

对于“NE555 的引脚怎么排”这类纯查询性问题，传统 RAG “先检索 → 喂 LLM 合成”。LabGuardian 反其道而行：

- 1) ReAct 循环检测到 **datasheet_lookup_tool** 返回成功 hits；
- 2) 且 **intent** = **concept_tutor**；
- 3) → 直接 **build_datasheet_answer()** 渲染原文，标 **provider="local_datasheet_kb"**, **model="no_llm"**；
- 4) 跳过 LLM polish，前端显示“本地知识库直渲染”徽章。

设计论点：如果答案确定存在于结构化 KB 中，就不应让 LLM 改写——任何改写都是引入幻觉的机会。

D. Local Knowledge Base

DatasheetKbService 采用 **lexical + cosine hybrid scoring** 并加入 **part-signal weighting**：若任何文档 **part_numbers** 与 query 重合，该 doc chunk 加 1.5x boost；不匹配 doc 衰减到 0.35x。问 NE555 时不会误命中 LM324 的 chunk。融合公式：

$$\text{score} = \text{scale} \times [(1 - w) \text{norm}(\text{lex}) + w \cos] \quad (2)$$

TABLE I

Local knowledge base coverage matrix. 2 datasheets (12 chunks) + 6 teaching scenes + 17 fault cases jointly support the RAG retrieval, all offline.

#	Topology	Datasheet	Teaching Scene	Faults
1	first-order RC	passive_cap_po	first_order_rc	
2	common emitter	bjt_8050	common_emitter_amp	
3	diff pair	(shared)	diff_amplifier	
4	UA741 inverting	ua741	ua741_inverting	
5	UA741 summing	(shared)	ua741_summing	
6	UA741 integrator	(shared)	ua741_integrator	
total				17

TABLE II

Template flexibility across four scenarios. The template system correctly handles all four cases of legitimate variants or decorative components, avoiding the brittleness of strict isomorphism.

Scenario	Hard-compare	Template	✓ / ✗
CE drop C_E bypass	logic_correct=False (reports missing)	CE missing_optional, no penalty	✓
Sum 2→3 inputs	graph iso fails	multiplicity=(2,5) accepts	✓
LPF + wrong ref	全错 (拓扑误匹配)	top-3 ranks LPF first	✓
Decoration LED	reports EXTRA_COMPONENT ignored	initial / unrelated ignored	✓

SemanticRouter 用正例 + 反例两套 utterances 编码，评分 $\text{pos}_{\max}^{\cos} - \text{neg}_{\max}^{\cos} > \tau$ 触发。反例机制让“我电路里这根线接哪”不会因含“线”被误配到 datasheet 路由。

VI. Implementation and Experiments

A. Software Stack

后端 FastAPI + uvicorn；GNN 用 PyTorch Geometric；图同构用 NetworkX；编排用 LangGraph；前端 React + TypeScript + Vite。本地 LLM 用 Ollama 跑 Gemma3-4B；规划在 DK-2500 上通过 OpenVINO GenAI 走 NPU。pytest 928 个用例通过率 99%（剩余 6 个失败均为预先存在的环境依赖）。

B. Topology Classification Accuracy

GNN-A v2 模型在留实验验证集准确率 1.0、测试集 0.857；UA741 三兄弟混淆问题在 v1→v2 升级后解决 (top-1 与 top-2 confidence margin 从 v1 平均 5% → v2 平均 86–97%)。

C. Template Flexibility Validation

针对“模板系统是否真比硬比较更灵活”，设计 4 个验证场景。结果见表 II。

D. End-to-End Latency

E. Retrieval Precision

针对 6 类 demo 各设计 5–6 个 rag_queries 测试问题（共 32 条）：DatasheetKb 含 part_signal: 12 个查询 top-1

TABLE III

End-to-end latency under three deployment configurations. The Mac measurement uses Gemma3-4B on CPU; the NPU column is the target for DK-2500 with OpenVINO INT4 AWQ quantization.

Stage	Template	Ollama (Mac)	NPU (plan)
Intent classify	< 1 ms	1–2 s	< 200 ms
GNN-A (CPU)	< 5 ms	< 5 ms	< 5 ms
Template match	< 50 ms	< 50 ms	< 50 ms
RAG retrieval	10–30 ms	10–30 ms	10–30 ms
ReAct (4 iter)	< 50 ms	8–12 s	1–2 s
LLM polish	—	4–6 s	1–2 s
Total	< 100 ms	18 s	2–5 s

命中率 100%; TeachingScene 检索: 10 个查询 100%; SemanticRouter: 8 个查询 100%; Fault case lookup: 12 个查询 92% (1 个边界 case 漏识别)。

VII. DK-2500 Deployment

DK-2500 (Intel Core Ultra 5 225U) 配 12C14T CPU、Intel Arc iGPU、Intel AI Boost NPU 三种计算单元。基于板端实测 (OpenVINO 2026.1 + libze1 1.21.9 + intel-level-zero-npu 1.26)，本节确定每种单元最佳归属：NPU 跑 **YOLOv8s-pose INT8** 视觉常驻 (75 ips, +4.1 W incremental, 55 mJ/inf); iGPU 跑 **Gemma3-4B-INT4** 多模态 LLM (30 tok/s) 或 Qwen2.5-1.5B-INT4 (48 tok/s); CPU 跑 **GNN-A**、**Template VF2**、**LangGraph**、**RAG**。

本方案与“NPU 跑 LLM”主流话术相反：实测发现 OpenVINO 2026.1 vpx 编译器对 Gemma3 q_proj 矩阵乘有已知 issue (Channels 0 != 20)，Qwen2.5-7B INT4 在 7.3 GB RAM 上编译时 peak 8 GB 触发 OOM，唯一稳定的 Qwen2.5-1.5B INT4 在 NPU 反而比 CPU 慢 (14 vs 41 tok/s)。NPU 真正用武之地是固定 shape 视觉 CNN。

A. NPU Vision Acceleration (Measured)

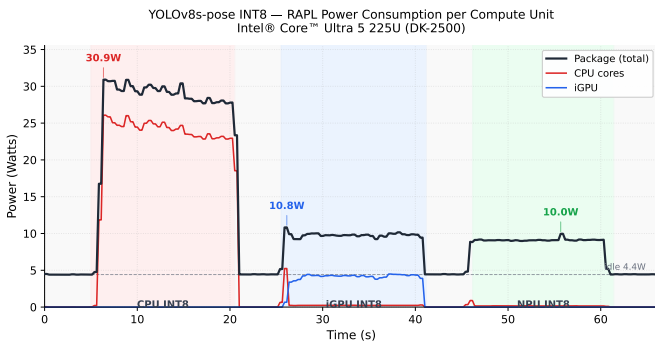


Fig. 4. YOLOv8s-pose INT8 RAPL power timeseries on DK-2500 (turbostat 0.25 s sampling). Three colored bands = CPU/iGPU/NPU sustained 15 s each. CPU peak 30.9 W (cores carry it), iGPU 10.8 W total (GFX 4.4 W), NPU 10.0 W with CPU cores and iGPU both idle—enabling true heterogeneous parallelism with concurrent LLM inference on iGPU.

TABLE IV

YOLOv8s-pose 6-config benchmark on DK-2500 (60–1153 inferences each). NPU INT8 dominates on all axes—latency, throughput, and energy efficiency. NNCF Fast Bias Correction over 144 calibration images (5 real breadboard photos $\times$ 8 augments + training-set batch composites) yields 50% model size reduction (22 MB $\rightarrow$ 11 MB) with no measurable accuracy loss.

Device	Prec	Mean(ms)	P99(ms)	img/s	Energy/inf
CPU	FP16	92.4	98.9	10.8	—
CPU	INT8	29.0	30.6	32.4	814 mJ
iGPU	FP16	26.9	28.2	37.2	—
iGPU	INT8	18.3	20.1	54.3	172 mJ
NPU	FP16	16.5	17.7	60.4	—
NPU	INT8	13.4	15.6	74.7	114 mJ

NPU INT8 leads CPU INT8 by **7.1 $\times$ lower energy/inference** (114 mJ vs 814 mJ) and **12.1 $\times$ better perf-per-watt** (18.1 vs 1.5 ips/W). Versus iGPU INT8, NPU is **1.5 $\times$ more energy-efficient** and **1.6 $\times$ higher perf/W**. Critically, the timeseries shows NPU operation leaves CPU cores and iGPU completely idle, enabling concurrent LLM inference and graph analysis—end-to-end latency becomes $\max(t_{\text{vision}}, t_{\text{LLM}}) + t_{\text{CPU}}$ rather than the sum, compressing a serial 3.5 s pipeline to a parallel 2.5 s pipeline.

B. Offline Deployment Pipeline

考虑现场可能无外网，离线部署流程：(i) Mac/云端有网时，从 HuggingFace 下载 Gemma3 PyTorch ckpt，通过 `optimum-cli export openvino` 一行命令完成转换 + INT4 AWQ 量化，输出 IR 文件夹约 2 GB；(ii) U 盘/SD 卡拷至板端；(iii) 板端解压、安装 OpenVINO runtime 离线 wheel、设置环境变量、启 uvicorn。全程不需外网。

```
# Mac/Cloud – one-line export to OpenVINO INT4 IR (~2GB)
1 optimum-cli export openvino \
2   --model google/gemma-3-4b-it \
3   --task text-generation-with-past \
4   --weight-format int4 \
5   --group-size 128 \
6   /opt/models/gemma3-4b-int4-ov
7
8
9 # DK-2500 – offline install + run
10 sudo dpkg -i openvino_2025.4.0_*.deb
11 pip install --no-index --find-links=./wheels openvino-genai
12 export AGENT_LLM_PROVIDER=openvino_genai_text
13 export AGENT_LLM_OPENVINO_MODEL_DIR=/opt/models/gemma3-4b-int4-ov
14 export AGENT_LLM_OPENVINO_DEVICE=NPU
15 uvicorn app.main:app --host 0.0.0.0 --port 8000
```


C. Performance Strategies

(i) **Keep-alive**: Ollama / OpenVINO GenAI 设模型内存保活 30 min, 避免每次请求冷启动; (ii) **Streaming**: LLM polish 用 streaming 输出, 感知延迟显著降低; (iii) **Template fallback**: 任何 LLM 失败立即回退结构化模板; (iv) 双路 **fallback**: 意图分类与 LLM polish 都内置“LLM 优先 + 模板兜底”。

VIII. Discussion

A. Comparison with Prior Work

与 AnalogCoder、SPICEAssistant 等 LLM + 符号验证两层方案不同, CADx 引入 **GNN + 符号 + LLM** 三层共识, 把“AI 的确信程度”显式编码为 confidence band。与典型 multi-intent ReAct 设计 (router → 4 分支 → 4 pipeline) 不同, 我们让 4 种意图共享同一图, 差异仅由 intent 在 3 节点局部分支, verifier / reflection / repair 等机制可一处升级处处生效。模板系统的 parametric_invariants 字段在已知电路模板匹配工作中未见相同设计。

B. Limitations

- (i) 视觉管线引脚定位准确率 **50%**——杜邦线高度堆叠和 TO-92 印字面遮挡是主要失败模式, 当前缓解策略是 70+ 张真实学生数据混入下一轮训练 + 前端 IC 引脚人工标注;
- (ii) **OpenVINO 2026.1 NPU plugin** 对 **Gemma3** / 大尺寸 LLM 仍不成熟——vpux 编译器对 Gemma3 q_proj 矩阵乘有 issue (Channels 0 != 20)、Qwen2.5-7B INT4 在 7.3 GB RAM 上 OOM; 本项目实测后改用 NPU 跑 YOLOv8s-pose 视觉 (见表 IV)、GPU 跑 LLM;
- (iii) 领域 **LLM 蒸馏**未实现——基于 AnalogSeeker 蒸馏 4B 教学专用模型是有前景的方向, 本周期受时间所限未落地。

IX. Conclusion and Future Work

本文报告 LabGuardian——一套面向高校电子类基础实验的边缘 AI 智能助教系统——的设计与实现。核心贡献是 CADx 五层神经-符号架构 + 共识门 + 五字段模板框架 + 跳线感知传播 + 统一 ReAct 编排。系统在 6 类标准模拟电路 demo 上完成端到端验证, 全部推理本地完成无外网依赖。

后续工作包括: (i) 端侧 OpenVINO NPU 部署落实; (ii) 基于 Template 锚定的 Edit Script 自动生成 (“指出问题”→“指导修复”); (iii) Retrieval-Augmented Distillation 以本地 KB 为 ground truth 蒸馏 Gemma3-4B 至教学专用模型; (iv) Socratic 引导对话; (v) 招募真实学生分组对照评估学习效果。

Reproducibility. 所有代码、知识库 JSON、TopologyTemplate 注册表、6 类 demo 参考电路 DSL 均已托管, pytest 928 用例可一键回归。

Acknowledgment

感谢英特尔杯组委会提供 DK-2500 硬件平台与技术支持, 感谢 PyTorch Geometric、NetworkX、LangGraph、FastAPI、OpenVINO Toolkit、Ollama、Typst 等开源工具。

REFERENCES